Blockchain Practical Guide: Summary and Analysis

BE (Computer Engineering) 2019 Pattern | Practicals 1 to 6

Summary

This summarises the full 26-page PDF guide (Blockchain_Technology_Practical_Guide.pdf). I could not run a Solidity compiler in my environment, so I checked both contracts by reading every line. Remix will compile them in about 10 seconds, and that compile is the final proof.

Are the codes correct?

Yes. Use `03_BankAccount.sol` and `04_StudentData.sol` exactly as they are. The logic and syntax are valid for Solidity 0.8.20 or newer, so no code changes are needed. For Practicals 1 to 4 you also do not need to install Python, Node.js, Ganache or a Solidity compiler. You only need a browser, MetaMask and the Remix website.

Small things to know (none stops the programs)

- **Wrong comment in the bank file (lines 17 to 19).** It says the receiver gets only a 2300 gas stipend. That is wrong for this code, because `.call` forwards gas. The code is still safe, because the balance is updated before the ETH is sent. Mention this correction if the examiner asks.
- **`transferTo` only moves the internal ledger balance.** It does not send ETH to your friend's wallet. They must call `withdraw` to receive the ETH.
- **`removeStudent` can change the order of students.** It moves the last student into the removed slot (swap and pop). This is intended.

What the PDF covers

- **Analysis:** a verdict on each file and a direct answer on whether you can run them as they are.

- **Theory first:** blockchain, hashing, consensus, Ethereum, smart contracts, gas, wallets and test networks.
- **Ubuntu lab setup:** what to open and where, plus a fallback (Remix VM) if the lab blocks MetaMask.
- **Practicals 1 and 2:** MetaMask install, wallet creation, Sepolia, faucet.
- **Practicals 3 and 4:** concept tables, part-by-part code walkthroughs, worked number examples, exact Remix click-steps, test tables with expected outputs, common errors and viva questions.
- **Gas and fee worksheet:** where to find each number, with a worked calculation.
- **Practicals 5 and 6, exam-day plan and glossary:** theory, the exam-day order of work, a troubleshooting table, 30 viva questions with answers, and the glossary.

Two things to watch in Remix

1. **Set VALUE back to 0** after depositing or sending ETH. Otherwise `withdraw`, `transferTo` and the student functions fail instantly.
2. **Check the network says Sepolia** before every deploy.

Limits

The files for Practicals 1, 5 and 6 were not in the upload. Those sections come from the README plus standard theory, and the PDF flags them. The Hyperledger commands are the usual Fabric `test-network` ones, so follow the folder's own README first. The gas numbers in the worksheet are sample values only, so record your own real ones.

Do this first

Do Practical 2 first: create the wallet and get Sepolia test ETH, since everything else depends on it.

FULL GUIDE

Everything below is the complete content of the 26-page PDF, in order. Practicals 1, 5 and 6 files were not uploaded, so those parts follow your README plus standard theory.

Part 0: Analysis of your uploaded files

I read `README.md`, `02_CreateWallet.md`, `03_BankAccount.sol` and `04_StudentData.sol` line by line. I could not run a Solidity compiler in my sandbox, so the verdict comes from careful reading. Remix will compile the files in 10 seconds when you paste the code, and that compile is your final proof.

File	Verdict	Details
`03_BankAccount.sol`	Correct. Use as it is.	Valid for Solidity 0.8.20+. Balance is reduced before ETH is sent (prevents re-entrancy). Only one comment is wrong.
`04_StudentData.sol`	Correct. Use as it is.	Struct, array, mapping, receive() and fallback() are valid. Swap-and-pop removal correctly updates the moved student's index.
`02_CreateWallet.md`	Good instructions.	Just follow them. MetaMask menu names change slightly between versions.
`README.md`	Useful.	Lists six practicals and the test cases used in this guide.

Things to know (none stops the program)

1. **Wrong comment in 03_BankAccount.sol (lines 17 to 19).** It says withdraw gives the receiver only a 2300 gas stipend. That is false here. `.call{value: amount}("")` forwards almost all remaining gas. The 2300 stipend applies to the old `transfer()` and `send()`. The code is safe because the balance is updated BEFORE the call. Say this correction in the viva if asked.
2. **transferTo() moves only the internal bank balance.** It does not send ETH to the classmate's MetaMask wallet. Your friend must call `withdraw` to receive ETH.
3. **`private` does not mean secret.** `balances` is private, so other contracts cannot read it, but anyone can read blockchain storage. That is why `balanceOf()` exists.
4. **removeStudent changes the order.** The last student moves into the removed slot, so `getAllStudents` order can change. This keeps removal cheap.
5. **Student id 0 is allowed** because the mapping stores position + 1. Use ids 1, 2, 3 in the exam to avoid confusion.

6. **Compiler version.** Both files say `pragma solidity ^0.8.20;`, so any compiler 0.8.20 or newer works. If deployment fails with an invalid-opcode style error, open Solidity Compiler, Advanced Configurations, set EVM version to `paris` or `shanghai`, compile and deploy again.

> **Exam point:** Practicals 3 and 4 can be copied exactly as they are into Remix. You do not edit anything. You do not install Python, Node.js, Ganache or a Solidity compiler for Practicals 1 to 4. You only need a browser, MetaMask and the Remix website.

Part 1: Theory you must know first

1.1 What is a blockchain?

A blockchain is a shared digital record book (ledger) copied on thousands of computers called nodes. New records are grouped into blocks. Each block contains the hash of the previous block, so all blocks are chained. If someone changes an old record, its hash changes, the next block's stored hash no longer matches, and the chain after it becomes invalid. This makes the record tamper-evident (immutable).

Property	Meaning in simple words
Decentralised	No single bank or company owns the ledger. Many nodes hold a copy.
Immutable	Old data practically cannot be changed without everybody noticing.
Transparent	On public chains anyone can see transactions (addresses, not real names).
Consensus-based	Nodes follow rules to agree which block is the next valid one.
Cryptographically secure	Hashing and digital signatures protect data and prove who sent a transaction.

Structure of a block

- **Block header:** previous block hash, Merkle root (one hash summarising all transactions), timestamp, nonce (used in Proof of Work), block number.

- **Block body:** the list of transactions.
- **Hash:** a fixed-size fingerprint of data (SHA-256 in Bitcoin, Keccak-256 in Ethereum). Same input gives the same hash; a tiny change gives a completely different hash; you cannot go backwards from hash to data.

Consensus: how do nodes agree?

Mechanism	How it works	Used by
Proof of Work (PoW)	Miners spend electricity solving a hard puzzle. First to solve adds the block.	Bitcoin (and Ethereum before Sept 2022)
Proof of Stake (PoS)	Validators lock (stake) ETH. A validator is chosen to propose a block; cheating loses stake.	Ethereum today (since The Merge, Sept 2022), so Sepolia also uses PoS
Raft / PBFT (permissioned)	A known set of nodes vote. Fast, no mining.	Hyperledger Fabric (Raft ordering service)

1.2 What is Ethereum?

Ethereum is a public blockchain platform that runs programs called smart contracts, not just coin transfers. Its coin is Ether (ETH). It has a global virtual computer called the EVM (Ethereum Virtual Machine). Every node runs the same contract code and gets the same result.

Two kinds of accounts

	EOA (Externally Owned Account)	Contract Account
Controlled by	A person with a private key (your MetaMask account)	Its own code. No private key.
Can start a transaction?	Yes	No. It only runs when called.
Has code?	No	Yes (the deployed smart contract)
Example	`0x8f3A...c21B` (your wallet)	The address you get after clicking Deploy in Remix

1.3 What is a smart contract?

A smart contract is a program stored on the blockchain that runs automatically when called. Once deployed, its code cannot be changed and everyone can verify what it does. Example: the Bank contract holds users' balances and enforces rules (you cannot withdraw more than you have).

- **Solidity** is the language for writing Ethereum smart contracts (similar to JavaScript/C++).
- **Compile:** Solidity source is converted into bytecode (what the EVM runs) and an ABI (a description of the functions, used by Remix to create buttons).
- **Deploy:** sending a special transaction that stores the bytecode on the blockchain. The contract gets an address.
- **Interact:** calling its functions. A read (view) call is free. A write call is a transaction that costs gas.

1.4 Transactions, gas and fee (very important for Practical 4)

Every operation that changes blockchain state needs computation by thousands of nodes, so the sender pays a fee called gas. Gas measures work. Storing new data costs the most (it is kept forever); simple arithmetic costs little.

Term	Meaning
Gas used	Actual units of work the transaction consumed.

Term	Meaning
Gas limit	Maximum gas you allow. Unused gas is refunded. If the work needs more, the transaction fails (out of gas) but the used gas is still charged.
Gas price	Price per gas unit, written in gwei.
Base fee + priority fee	Since EIP-1559: price = base fee (set by the network, burned) + priority tip (goes to the validator).
Transaction fee	Fee = Gas used × Gas price. With the price in gwei: Fee (ETH) = Gas used × Gas price(gwei) ÷ 1,000,000,000.

Unit	Value
1 wei	Smallest unit
1 gwei	1,000,000,000 wei = 10^9 wei
1 ether	1,000,000,000,000,000,000 wei = 10^18 wei
0.5 ether	500,000,000,000,000,000 wei (`500000000000000000`)

Why numbers appear in wei: Solidity has no decimals, so all money values are whole numbers in wei. Remix lets you pick the unit (wei, gwei, ether) next to the Value box.

1.5 Wallet, keys and address

- **Private key:** a secret 256-bit number that signs your transactions. Anyone with it controls your funds.
- **Public address:** derived from the private key (`0x` + 40 hex characters). Safe to share, like a bank account number.
- **Secret Recovery Phrase (seed phrase):** 12 words that can regenerate all your private keys. Never share it, never screenshot it, never type it into a website.
- **Digital signature:** proof that the owner of the private key authorised a transaction, without revealing the key.

MetaMask is a browser extension wallet. It stores your keys (encrypted on your computer), shows balances, and asks you to confirm each transaction.

1.6 Networks: Mainnet, Testnet, Sepolia

Network	Real money?	Use
Ethereum Mainnet	Yes, real ETH	Real applications. NOT for practicals.
Sepolia testnet	No. Test ETH is free and worthless	Practicals and development. Free ETH comes from a faucet.
Remix VM (in-browser)	No. Fake accounts with 100 fake ETH	Quick offline testing. No MetaMask needed.

1.7 Tools you will use

Tool	What it is	Where you use it
MetaMask	Wallet extension in the browser	Browser toolbar (fox icon)
Remix IDE	Online editor + compiler + deployer for Solidity	Browser tab: remix.ethereum.org
Faucet	Website that gives free test ETH	Browser
Sepolia Etherscan	Public site to look up any transaction, address or contract	Browser: sepolia.etherscan.io

Part 2: Lab setup on Ubuntu (step by step)

Everything for Practicals 1 to 4 happens inside a web browser. You use the Terminal only to check things. Nothing needs installing with commands unless the lab PC has no browser.

2.1 Where will I do what?

Task	Where
Create wallet, switch to Sepolia, request test ETH	Browser + MetaMask extension

Task	Where
Write or paste Solidity, compile, deploy, test functions	Browser tab with Remix IDE
Check Fabric tools (Practical 6 only)	Ubuntu Terminal (Ctrl + Alt + T)
Take screenshots for your journal	PrtSc (whole screen) or Shift + PrtSc (select area); images go to the Pictures folder

2.2 Get the browser ready

1. Log in to the Ubuntu computer with the lab credentials.
2. Click the **Firefox** icon in the left dock. (Or press the Super/Windows key, type `firefox`, press Enter.)
3. Check internet: open `https://remix.ethereum.org`. If it loads, internet works. Faucets, MetaMask and Remix all need internet.
4. Optional terminal check: press Ctrl + Alt + T, type `ping -c 3 google.com`, press Enter. Replies with time values mean internet is fine. Press Ctrl + C to stop.
5. If your lab has Chromium or Chrome instead, that is fine. MetaMask supports Chrome, Brave, Edge and Firefox.

> **Tip:** If the lab has no browser at all, ask the lab assistant. With admin rights, `sudo snap install chromium` installs Chromium (needs internet and a sudo password). Do not try this without permission.

2.3 What if the lab blocks installing extensions?

Ask your lab incharge first. If MetaMask truly cannot be installed, use this fallback:

- In Remix, Deploy & Run Transactions, Environment = **Remix VM** (the name has a fork like Cancun or Prague). You get test accounts instantly and can run Practicals 3 and 4 completely. Gas values show, but there is no real Sepolia transaction hash or Etherscan page.
- Tell your examiner clearly that you used Remix VM because the extension could not be installed. Explain Practicals 1 and 2 using screenshots from a home PC or phone.

2.4 Prepare BEFORE exam day

1. Create the wallet and get Sepolia ETH at home or in a free lab slot. Faucets can be slow or limited per day.
2. Keep `03_BankAccount.sol`, `04_StudentData.sol` and the README on a pen drive, email or cloud drive.
3. Compile both contracts once at home in Remix so there are no surprises.
4. Write your recovery phrase on paper only. Do not bring it in a phone photo.

> **Careful:** Lab computers are shared. After finishing, in MetaMask click Lock, or remove the extension if the lab permits, and clear the browser data. Use a test-only wallet; never import a wallet that has real money.

Part 3: Practical 1, Install MetaMask and study Ether spent per transaction

> `01_MetaMaskInstallation.md` was not uploaded. This follows the README line and standard MetaMask steps. If your own file has extra steps, follow those too.

3.1 Theory

MetaMask is a browser extension (small add-on) that works as a crypto wallet and as a bridge between websites (like Remix) and the Ethereum network. It injects an object called `window.ethereum` into web pages; Remix uses it when you choose **Injected Provider - MetaMask**.

3.2 Install on Ubuntu (Firefox or Chrome/Chromium)

1. Open the browser and go to `https://metamask.io`. Always start from this official site so you do not install a fake copy.
2. Click **Download**. The site detects your browser and shows Install MetaMask for Firefox (or Chrome).
3. You land on the official add-on store page. Check the publisher/name says MetaMask.
4. Click **Add to Firefox** (or Add to Chrome). A permission popup appears. Click **Add**.
5. A fox icon appears in the toolbar. If not visible, click the puzzle-piece (Extensions) icon and pin MetaMask.

6. A new tab opens with the MetaMask welcome page. Continue with Practical 2.

3.3 Study: Ether spent per transaction

1. After you have Sepolia test ETH (Part 4), click **Send** in MetaMask.
2. Paste a classmate's address (or a second account of yours), enter `0.001`, click **Next**, then **Confirm**.
3. The confirm screen shows the estimated gas fee. For a plain ETH transfer the gas is always **21,000**.
4. Open the **Activity** tab, click the transaction, click **View on block explorer / Etherscan**.
5. On Sepolia Etherscan note: Value (ETH sent), Transaction Fee (ETH spent on gas), Gas Price, Gas Limit & Usage, Status, Block.
6. Verify the maths: Fee = 21,000 × gas price (gwei) ÷ 10^9.

Observation to write in journal	Example (numbers will differ)
Gas used for simple ETH transfer	21,000
Gas price	1.5 gwei
Fee in ETH	21,000 × 1.5 ÷ 10^9 = 0.0000315 ETH
Total spent	Amount sent + fee

> **Exam point:** Why is the fee for sending ETH always 21,000 gas? It is the fixed base cost of any transaction (intrinsic gas). Smart-contract calls cost more because they run code and write storage.

Part 4: Practical 2, Create your own wallet with MetaMask

This follows your `02_CreateWallet.md`. Menu names may differ a little in the newest MetaMask version, but the idea is the same.

4.1 Create the wallet

1. Open MetaMask (fox icon) or the welcome tab. Click **Get started**.
2. Choose **Create a new wallet**. Do NOT choose Import (import is for restoring an old wallet).

3. If asked about analytics, you may decline.
4. Set a **password** (minimum 8 characters). It protects MetaMask on this computer only. It is not your recovery phrase.
5. Click **Secure my wallet**. MetaMask shows the Secret Recovery Phrase (12 words).
6. Write all 12 words in order on paper. Number them 1 to 12. Do not take a photo.
7. Click Continue and confirm by selecting/typing the requested words in order.
8. Finish. Your first account `Account 1` opens with a public address like `0x8f3A...c21B`.

> **Careful:** If you lose the 12 words and lose the computer, the wallet is gone forever. MetaMask has no helpline to recover it. If anyone else sees the words, they can take everything.

4.2 Switch to the Sepolia test network

1. Click the network dropdown at the top of MetaMask (it says Ethereum Mainnet or similar).
2. If Sepolia is not listed, turn on **Show test networks** in the dropdown, or in Settings, Advanced.
3. Click **Sepolia**. The balance reads 0 SepoliaETH (balance is separate on every network).

4.3 Get free test ETH from a faucet

1. In MetaMask copy your public address (click the account name or copy icon).
2. Open a faucet. Your README lists `https://cloud.google.com/application/web3/faucet/ethereum/sepolia` and `https://sepoliafaucet.com`.
3. Paste your address, solve any captcha or log in if asked, and click request/send.
4. Wait about 1 minute and check MetaMask. You should see something like 0.05 SepoliaETH.

> **Tip:** Faucets sometimes have daily limits, need a login, or are down. Try the other faucet, try later, or ask a classmate to send a small amount (0.05 ETH is enough for the whole practical).

4.4 Understand what you see

Item	Meaning
Public address	Starts with `0x`, 40 hex characters. Share it to receive funds.
Network selector	Which chain you are on. Always confirm it says Sepolia before signing.
Balance	ETH on the selected network only.
Activity tab	Your transaction history.
Account details	Shows address and QR. Export private key exists but avoid using it.

4.5 Backup and restore (viva)

- **Backup:** Settings, Security & privacy, Reveal Secret Recovery Phrase (asks password). Write on paper.
- **Restore:** on another computer choose Import an existing wallet, enter the 12 words, set a new local password. All accounts reappear because they derive from the phrase (HD wallet).
- **Add account:** account menu, Add account. New address, same recovery phrase.

4.6 Verify your wallet works

- Address starts with `0x` and has 40 hex characters.
- Network = Sepolia.
- Balance shows test ETH.
- A test send of 0.001 ETH appears in Activity with status Confirmed.

Part 5: Practical 3, Bank Account smart contract

5.1 Aim and what the program does

Aim: Write a Solidity smart contract for a bank account where customers can deposit, withdraw and check their balance, and deploy it on Sepolia.

Think of the contract as a tiny bank branch on the blockchain. It keeps a ledger (a mapping) of how many wei each customer has. When you deposit, real (test) ETH goes into the contract and your ledger entry increases. When you withdraw, your ledger entry decreases first and then ETH is sent to your wallet.

Function	What it does	Costs gas?
`deposit()`	Receives ETH you send with the call and adds it to your balance	Yes (write, payable)
`withdraw(amount)`	Sends `amount` wei back to you if you have enough balance	Yes (write)
`transferTo(recipient, amount)`	Moves part of your internal balance to another customer's internal balance	Yes (write)
`balanceOf(account)`	Shows one customer's balance in wei	No (view)
`contractBalance()`	Shows total ETH physically held by the contract	No (view)
`owner()`	Shows the address that deployed the contract	No (view)
`totalDeposits()`	Shows the sum of all customer balances	No (view)
`transferOwnership(newOwner)`	Owner hands the contract to a new owner	Yes (write, owner only)

5.2 Solidity concepts used (learn these first)

Concept	Simple explanation	Where in this code
`// SPDX-License-Identifier`	A comment stating the software licence. Compiler warns if missing.	Line 1
`pragma solidity ^0.8.20;`	Compile with 0.8.20 or newer (below 0.9.0).	Line 2

Concept	Simple explanation	Where in this code
`contract`	Like a class: holds data and functions.	`contract BankAccount`
State variable	Stored permanently on the blockchain (costs gas to change).	`owner`, `balances`, `totalDeposits`
`mapping(address => uint256)`	Key-value table. Looks up a number by address. Unset keys return 0.	`balances`
`uint256`	Unsigned 256-bit integer (0 to about 10^77). Cannot be negative.	Balances, amounts
`address`	A 20-byte Ethereum address.	`owner`, `recipient`
`msg.sender`	Address that called the function right now.	deposit, withdraw
`msg.value`	Amount of wei sent with the call.	deposit
`payable`	Marks a function or address that may receive ETH.	`deposit() external payable`
`external` / `public`	external: callable from outside. public: also from inside, and public variables get an automatic getter.	functions; `owner`
`view`	Function only reads data. Free when called directly.	`balanceOf`
`event` / `emit`	Writes a log entry apps and Etherscan can read. Cheaper than storage.	`Deposited`, `Withdrawn`
`indexed`	Makes an event field searchable by filters.	`address indexed account`

Concept	Simple explanation	Where in this code
`error` / `revert`	Custom error that cancels the transaction and undoes all changes. Cheaper than long text messages.	`InsufficientBalance`
`modifier`	Reusable check that runs before a function body. `_;` marks where the body runs.	`onlyOwner`
`constructor`	Runs once, at deployment.	sets `owner`
`.call{value: x}("")`	Low-level way to send ETH. Returns a success flag.	`withdraw`
`address(this).balance`	Real ETH held by this contract.	`contractBalance`

5.3 Code walkthrough

(a) Header and state variables

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {
    address public owner;                           // who deployed the
contract
    mapping(address => uint256) private balances;  // customer -> balance
(wei)
    uint256 public totalDeposits;                   // sum of all customer
balances
```

`owner` is public, so Solidity auto-creates an `owner()` button in Remix. `balances` is private, so we need `balanceOf()` to read it. `totalDeposits` tracks the combined balance.

(b) Events, errors, modifier, constructor

```
    event Deposited(address indexed account, uint256 amount, uint256
newBalance);
    event Withdrawn(address indexed account, uint256 amount, uint256
newBalance);
    event Transferred(address indexed from, address indexed to, uint256
amount);
    event OwnershipTransferred(address indexed previousOwner, address indexed
newOwner);

    error NotOwner();
    error ZeroAmount();
    error InsufficientBalance(uint256 requested, uint256 available);
    error InvalidRecipient();
    error TransferFailed();

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }

    constructor() {
        owner = msg.sender;      // the account that clicks Deploy becomes
owner
    }
```

Events are the receipts you see in the Remix terminal. Errors give clear failure reasons: `InsufficientBalance(requested, available)` tells you both numbers. The modifier `onlyOwner` stops everybody except the owner. The constructor runs once, at deployment, and stores the deployer as owner.

(c) deposit()

```
    function deposit() external payable {
        if (msg.value == 0) revert ZeroAmount();

        balances[msg.sender] += msg.value;
        totalDeposits += msg.value;

        emit Deposited(msg.sender, msg.value, balances[msg.sender]);
    }
```

Because it is `payable`, the ETH you attach (the Value box in Remix) moves into the contract. The code rejects a zero deposit, increases your ledger entry and the total, then emits the receipt event.

(d) withdraw(), the most important function

```
    function withdraw(uint256 amount) external {
        if (amount == 0) revert ZeroAmount();

        uint256 current = balances[msg.sender];
        if (amount > current) revert InsufficientBalance(amount, current);
// CHECKS

        balances[msg.sender] = current - amount;
// EFFECTS
        totalDeposits -= amount;

        (bool ok, ) = payable(msg.sender).call{value: amount}("");
// INTERACTION
        if (!ok) revert TransferFailed();

        emit Withdrawn(msg.sender, amount, balances[msg.sender]);
    }
```

This follows the **Checks, Effects, Interactions** pattern: (1) check conditions, (2) update your own records, (3) only then send money outside. If the receiver is a malicious contract, receiving ETH triggers its code, which could call `withdraw` again before the first call finished. This attack is called **re-entrancy** (the famous 2016 DAO hack was this type). Because the balance is already reduced before the ETH leaves, the second call sees a smaller balance and fails.

(e) transferTo()

```
    function transferTo(address payable recipient, uint256 amount) external {
        if (recipient == address(0)) revert InvalidRecipient();
        if (amount == 0) revert ZeroAmount();

        uint256 current = balances[msg.sender];
        if (amount > current) revert InsufficientBalance(amount, current);

        balances[msg.sender] = current - amount;
        balances[recipient] += amount;

        emit Transferred(msg.sender, recipient, amount);
    }
```

Moves ledger balance between two customers inside the contract. `address(0)` is the empty address (`0x000...0`); sending to it would burn money, so it is blocked. No ETH leaves the contract here, so `contractBalance` does not change.

(f) View functions and ownership

```
    function balanceOf(address account) external view returns (uint256) {
        return balances[account];
    }

    function contractBalance() external view returns (uint256) {
        return address(this).balance;
    }

    function transferOwnership(address newOwner) external onlyOwner {
        if (newOwner == address(0)) revert InvalidRecipient();
        address previous = owner;
        owner = newOwner;
        emit OwnershipTransferred(previous, newOwner);
    }
}
```

`balanceOf` reads the ledger. `contractBalance` reads the real ETH held. They should be equal unless someone force-sends ETH. `transferOwnership` can only be called by the current owner (modifier).

5.4 Worked example: how numbers move

Step	Action	Your balance	Classmate balance	Contract holds
1	Deploy	0	0	0
2	deposit with Value 0.5 ether	0.5 ETH	0	0.5 ETH
3	withdraw 200000000000000000 (0.2 ETH)	0.3 ETH	0	0.3 ETH
4	transferTo(classmate, 100000000000000000)	0.2 ETH	0.1 ETH	0.3 ETH
5	withdraw 999000000000000000	reverts (InsufficientBalance)	-	-

In step 4 the contract still holds 0.3 ETH; only the ledger changed. The failed transaction in step 5 still uses some gas but changes nothing.

5.5 How to run it (exact steps)

Step 1: Open Remix

1. In a new browser tab go to `https://remix.ethereum.org`. Wait for it to load.
2. The left icon bar: top icon = File Explorer. Below it: Search, Solidity compiler, Deploy & run transactions.
3. File Explorer shows a `contracts` folder with sample files. Ignore them.

Step 2: Put the code in Remix (choose ONE way)

Way A (copy-paste):

1. Click the **Create new file** icon (page with plus) in File Explorer. Type `BankAccount.sol`, press Enter. The file opens in the editor.
2. On Ubuntu, open your `03_BankAccount.sol` (double-click; it opens in the Text Editor). Press Ctrl + A, then Ctrl + C.
3. Click inside the Remix editor and press Ctrl + V. Make sure the first line is `// SPDX-License-Identifier: MIT`.

Way B (upload): In File Explorer click the **Upload / Load a local file** icon (upward arrow) and choose `03_BankAccount.sol`. The file name stays `03_BankAccount.sol`, which is fine. The contract inside is still called `BankAccount`.

Step 3: Compile

1. Click the **Solidity compiler** icon.
2. In Compiler choose a version that is 0.8.20 or newer (for example the newest 0.8.x).
3. Click the big blue button **Compile BankAccount.sol**.
4. Success = a green tick on the compiler icon. Errors appear in red. Yellow warnings are fine.

Step 4: Connect MetaMask and deploy

1. Click **Deploy & run transactions** (Ethereum-logo icon).
2. Under ENVIRONMENT choose **Injected Provider - MetaMask**. A MetaMask popup asks to connect. Select your account and click Next / Connect.
3. Remix now shows the network under the dropdown, e.g. `Sepolia (11155111)`, and your address in ACCOUNT. If it says Mainnet, STOP and switch MetaMask to Sepolia.
4. Make sure VALUE is `0` and the unit is Wei (for deployment).
5. In CONTRACT make sure `BankAccount` is selected.
6. Click the orange **Deploy** button. In MetaMask check the network says Sepolia and click **Confirm**.
7. Watch the Remix terminal (bottom). After 10 to 30 seconds you see a green tick. Under **Deployed Contracts** (bottom of left panel) your contract appears with its address. Click the small arrow to expand it and see the buttons.

> **Tip:** Copy the contract address and open `https://sepolia.etherscan.io/address/<your-contract-address>`. Show this page to the examiner as proof that your contract is live.

Step 5: Test the functions

Colour code: orange button = write transaction (costs gas, MetaMask pops up). Blue button = read (free).

#	Action in Remix	Expected result
1	Click **owner** (blue)	Your wallet address.

#	Action in Remix	Expected result
2	Set VALUE (top of the panel) to `0.5` and change the unit to Ether. Click orange **deposit**. Confirm in MetaMask.	Success. In the terminal details a `Deposited` log appears.
3	IMPORTANT: set VALUE back to `0`. In `balanceOf` paste your own address, click blue **balanceOf**.	`500000000000000000`
4	Click **contractBalance**	`500000000000000000`
5	In `withdraw` type `200000000000000000`, click orange **withdraw**. Confirm.	`Withdrawn` event. balanceOf now `300000000000000000`.
6	In `withdraw` type `999000000000000000` and click withdraw	Remix/MetaMask warns the transaction will fail. This is the expected `InsufficientBalance` revert. If you proceed it fails; cancelling is fine. Screenshot the warning/revert.
7	In `transferTo` type the classmate's address, a comma, and `100000000000000000`, like `0xAbC...123, 100000000000000000`. Click transferTo.	`Transferred` event. `balanceOf(classmate)` = `100000000000000000`.
8	Click **totalDeposits**	`300000000000000000`

> **Careful:** If you forget to set VALUE back to 0, calling a non-payable function such as `withdraw` or `transferTo` fails immediately, because those functions do not accept ETH. This is the number one beginner mistake.

Step 6: Record gas and fee

In the Remix terminal click the small arrow on a transaction line to expand it. Look for gas, transaction cost and execution cost (in gas units). Gas price and fee in ETH are in MetaMask's Activity details or on Etherscan. Use the worksheet in Part 7.

5.6 Common errors and fixes (Practical 3)

Problem	Reason	Fix
Compile error: Source file requires different compiler version	Chosen compiler is older than 0.8.20	Select 0.8.20 or newer and compile again
Deploy button does nothing	MetaMask popup hidden	Click the fox icon in the toolbar and confirm
Insufficient funds	No Sepolia ETH	Get from faucet; check you are on Sepolia
withdraw fails immediately	VALUE not zero, or amount larger than balance	VALUE = 0; check `balanceOf` first
transferTo fails with a format error	Address or number typed wrongly	Use `0xADDRESS, AMOUNT` and the exact 42-character address
Contract disappeared after refresh	Remix forgets deployed instances	Use **At Address**: paste your contract address and click At Address

5.7 Viva questions for Practical 3

- **Why is balance updated before sending ETH?** To block re-entrancy (Checks-Effects-Interactions).
- **msg.sender vs tx.origin?** msg.sender is the immediate caller; tx.origin is the original wallet that started the whole chain of calls. Using tx.origin for authorisation lets a malicious contract trick users (phishing). Use msg.sender.
- **Why mapping and not array?** Direct O(1) lookup by address. No loops, so cheap.
- **What does payable mean?** The function can receive ETH.
- **Difference between contractBalance() and balanceOf()?** The first is real ETH in the contract; the second is one customer's recorded share.
- **What happens in a failed transaction?** All state changes are undone (reverted). Gas that was used is still charged.
- **Why custom errors instead of require strings?** They cost less gas and can carry data.

- **Is private really private?** No. It only restricts other contracts. On-chain data is visible to anyone who reads storage.

Part 6: Practical 4, Student Data using structure, array and fallback

6.1 Aim and what the program does

Aim (from your file): Write a Solidity program to create Student data using structures, arrays and a fallback function. Deploy it on Ethereum and observe the transaction fee and gas values.

The contract is a small student database on the blockchain. Each record has id, name, course and age. You can add, view, update and delete students. It also has `receive()` and `fallback()` so you can see what happens when someone sends ETH or calls a function that does not exist.

Function	What it does	Gas?
`addStudent(id, name, course, age)`	Adds a new student. Fails if id exists or data is invalid.	Yes (most expensive)
`getStudent(id)`	Returns one student record. Fails if not found.	Free (view)
`getAllStudents()`	Returns every student in an array.	Free (view)
`studentCount()`	Returns how many students are stored.	Free (view)
`updateStudent(id, name, course, age)`	Changes name, course and age of an existing id.	Yes
`removeStudent(id)`	Deletes a student (swap with last, then pop).	Yes
`receive()`	Runs when plain ETH is sent with empty data. Adds to `totalDonations`.	Yes (transaction)
`fallback()`	Runs when a call matches no function. Stores the data and the sender.	Yes (transaction)

6.2 New concepts in this practical

Concept	Explanation
`struct`	Your own data type grouping related fields. Here `Student {id, name, course, age}` is like a table row or a C struct.
Array `Student[] students`	A dynamic list that grows with `push` and shrinks with `pop`. Index starts at 0. Reading all items costs more as the list grows.
Mapping `indexOf`	Finds a student's position by id instantly. It stores position + 1, so a stored 0 means not found (mappings return 0 for unknown keys).
`string`	Text data. Expensive to store. `calldata` means it is read directly from the incoming transaction without copying (cheaper).
`memory` / `storage` / `calldata`	storage = permanent blockchain data (expensive). memory = temporary during the call. calldata = read-only input data of the call.
`bytes`	Raw byte data. `msg.data` is the full calldata of the call.
Function selector	The first 4 bytes of calldata, identifying which function is called (first 4 bytes of the hash of the function signature).
`receive()`	Special function. Runs for plain ETH transfers with EMPTY calldata. Needs `external payable`.
`fallback()`	Special function. Runs when calldata is not empty but matches no function, or when empty and no `receive()` exists. Here it is `payable` so it also accepts ETH.

6.3 Code walkthrough

(a) Data structures

```
contract StudentData {
    struct Student {
        uint256 id;
        string  name;
        string  course;
        uint256 age;
    }

    Student[] private students;                  // all records
    mapping(uint256 => uint256) private indexOf; // id -> (position + 1)

    uint256 public totalDonations;     // total ETH received by
receive()/fallback()
    bytes   public lastFallbackData;   // calldata of the last fallback call
    address public lastFallbackSender; // who made the last fallback call
```

The array holds records; the mapping is a quick index. The three public variables prove that `receive` and `fallback` executed. Events (`StudentAdded` , `StudentUpdated` , `StudentRemoved` , `Received` , `FallbackReceived`) and errors (`StudentAlreadyExists` , `StudentNotFound` , `InvalidStudentData`) are declared next, like in Practical 3.

(b) addStudent

```
    function addStudent(uint256 id, string calldata name,
                        string calldata course, uint256 age) external {
        if (indexOf[id] != 0) revert StudentAlreadyExists(id);   // duplicate
id?
        if (bytes(name).length == 0 || bytes(course).length == 0 || age == 0)
{
            revert InvalidStudentData();                         // empty
name/course or age 0
        }

        students.push(Student(id, name, course, age));  // add at end of
array
        indexOf[id] = students.length;                  // store position+1
        emit StudentAdded(id, name, students.length - 1);
    }
```

The first two `if` lines are validation. `students.push(...)` appends a new struct (new storage, the costly part). `indexOf[id] = students.length` works because after pushing, the length equals the new position + 1. Example: first student, length becomes 1, so

`indexOf[1] = 1`, meaning position 0.

(c) Reading functions

```
    function getStudent(uint256 id) external view returns (Student memory) {
        uint256 slot = indexOf[id];
        if (slot == 0) revert StudentNotFound(id);
        return students[slot - 1];               // convert back to array
index
    }

    function getAllStudents() external view returns (Student[] memory) {
        return students;
    }

    function studentCount() external view returns (uint256) {
        return students.length;
    }
```

(d) updateStudent

```
    function updateStudent(uint256 id, string calldata name,
                           string calldata course, uint256 age) external {
        uint256 slot = indexOf[id];
        if (slot == 0) revert StudentNotFound(id);
        if (bytes(name).length == 0 || bytes(course).length == 0 || age == 0)
{
            revert InvalidStudentData();
        }

        Student storage student = students[slot - 1];  // a reference, not a
copy
        student.name = name;
        student.course = course;
        student.age = age;

        emit StudentUpdated(id, name);
    }
```

`Student storage student` is a pointer to the real stored record. Changing its fields changes blockchain data directly. The id never changes, so the mapping needs no update.

(e) removeStudent: swap and pop

```
    function removeStudent(uint256 id) external {
        uint256 slot = indexOf[id];
        if (slot == 0) revert StudentNotFound(id);

        uint256 position = slot - 1;
        uint256 lastPosition = students.length - 1;

        if (position != lastPosition) {
            students[position] = students[lastPosition];            // move
last into the hole
            indexOf[students[position].id] = position + 1;          // fix
moved student's index
        }

        students.pop();            // remove last element
        delete indexOf[id];        // forget removed id

        emit StudentRemoved(id);
    }
```

Deleting from the middle of an array normally means shifting every later item (slow, expensive). Trick: copy the last item into the removed spot, then `pop()` the last. Cost is constant (O(1)). Trade-off: order changes.

Step	students array (ids)	indexOf (id: position+1)
Start (added 1, 2, 3)	[1, 2, 3]	{1:1, 2:2, 3:3}
removeStudent(1): position 0, last 2	copy student 3 into position 0: [3, 2, 3]	indexOf[3] = 1
pop()	[3, 2]	{2:2, 3:1}
delete indexOf[1]	[3, 2]	{2:2, 3:1} (id 1 gone)

(f) receive and fallback

```
    receive() external payable {
        totalDonations += msg.value;
        emit Received(msg.sender, msg.value);
    }

    fallback() external payable {
        totalDonations += msg.value;
        lastFallbackData = msg.data;
        lastFallbackSender = msg.sender;
        emit FallbackReceived(msg.sender, msg.value, msg.data);
    }
}
```

You send...	Function that runs	Visible proof
ETH with empty calldata	`receive()`	`Received` event; `totalDonations` increases
ETH or no ETH with calldata `0xdeadbeef` (no such function)	`fallback()`	`FallbackReceived` event; `lastFallbackData` = `0xdeadbeef`; `lastFallbackSender` = your address
A real function call (e.g. addStudent)	That function	Normal behaviour

> **Exam point:** Difference between receive() and fallback()? receive() = plain ETH, empty calldata. fallback() = unknown function / calldata that matches nothing. A contract without payable receive/fallback REJECTS plain ETH transfers.

6.4 How to run it (exact steps)

1. Open `https://remix.ethereum.org`. MetaMask must be on **Sepolia**.
2. File Explorer: create a new file `StudentData.sol` (or upload `04_StudentData.sol`). Paste the full code from your file (Ctrl + A, Ctrl + C in the Text Editor; Ctrl + V in Remix).
3. Solidity compiler tab: version 0.8.20 or newer. Click **Compile StudentData.sol**. Wait for the green tick.
4. Deploy & run tab: Environment = **Injected Provider - MetaMask**. Check the network says Sepolia.

5. VALUE = 0. In CONTRACT choose `StudentData`. Click orange **Deploy** and **Confirm** in MetaMask.
6. Expand the contract under Deployed Contracts. You see orange and blue buttons plus a **Low level interactions** section at the bottom.
7. Note the deploy gas from the terminal. Now test using the table below.

6.5 Test cases with expected output

How to type inputs: click the small down-arrow next to a function to expand its separate boxes, or type everything comma-separated in the single box. **Strings need double quotes.**

#	Function and input	Expected result
1	`addStudent`: `1, "Asha", "Computer", 20`	Success. Log `StudentAdded` (id 1, name Asha, position 0).
2	`studentCount`	`1`
3	`getStudent`: `1`	`1, Asha, Computer, 20`
4	`addStudent`: `2, "Ravi", "IT", 21`	Success. Record the gas; it is lower than the first add.
5	`getAllStudents`	Array with both records.
6	`addStudent`: `1, "X", "Y", 19` (duplicate id)	Fails: `StudentAlreadyExists(1)`.
7	`addStudent`: `3, "", "IT", 20` (empty name)	Fails: `InvalidStudentData`.
8	`updateStudent`: `1, "Asha Patil", "IT", 21`	Success. `getStudent(1)` now shows `1, Asha Patil, IT, 21`.
9	`removeStudent`: `1`	Success. `studentCount` = 1. `getStudent(1)` now fails with `StudentNotFound`.
10	Receive test: VALUE `0.1` Ether. Low level interactions: leave CALLDATA empty, click **Transact**.	`Received` event. `totalDonations` = `100000000000000000`.

#	Function and input	Expected result
11	Fallback test: VALUE `0.1` Ether. CALLDATA `0xdeadbeef`. Click **Transact**.	`FallbackReceived` event. `lastFallbackData` = `0xdeadbeef`. `lastFallbackSender` = your address. `totalDonations` = `200000000000000000`.

> **Careful:** After the ETH tests, set VALUE back to 0 before calling addStudent, updateStudent or removeStudent. Those are not payable and fail if VALUE is not zero.

> **Tip:** Reads are free (blue buttons). Writes ask MetaMask for confirmation each time: add, update, remove, Transact. Confirm each one and wait for success before the next.

6.6 Why is the first addStudent so expensive?

Writing a new storage slot costs about 20,000 gas each. A student has several storage slots (id, two strings, age) plus the mapping entry and array length update, so the first add costs far more than reading or updating. A later add is a bit cheaper because some storage is already set up. Updating existing slots is cheaper than creating new ones, and deleting can give a partial gas refund. A view call costs nothing because it does not change the blockchain.

6.7 Common errors (Practical 4)

Problem	Fix
String parameter error	Use double quotes: `"Asha"`, not single quotes or none
Transaction fails on addStudent	VALUE must be 0; also the id may already exist
Fallback not triggering	CALLDATA must be non-empty hex starting with `0x` and must not match a real function selector
Plain ETH triggers receive, not fallback	Correct behaviour: empty calldata always goes to receive()
`getStudent` reverts	Id not present, or already removed

Part 7: Gas and transaction fee worksheet

Your README asks you to record gas used, gas price and fee for several operations. Below is how to find each number and a filled sample so you understand the calculation. **Your real numbers will differ. Write your own.**

7.1 Where to find the numbers

Number	Where
Gas used	Remix terminal: expand the transaction; also Etherscan, Gas Limit & Usage by Txn
Gas price (gwei)	Etherscan Gas Price (or Effective gas price); also MetaMask Activity, transaction details
Fee in ETH	Etherscan Transaction Fee (to cross-check your own calculation)

7.2 Sample worked example (illustrative numbers only)

Operation	Gas used	Gas price (gwei)	Fee (ETH) = gas × price ÷ 10^9
Deploy StudentData	1,100,000	1.5	0.00165
addStudent (first)	130,000	1.5	0.000195
addStudent (second)	105,000	1.5	0.0001575
updateStudent	55,000	1.5	0.0000825
removeStudent	45,000	1.5	0.0000675
receive (plain ETH)	28,000	1.5	0.000042
fallback (0xdeadbeef)	75,000	1.5	0.0001125

> These sample numbers only show the arithmetic. Do not copy them into your journal. Real values depend on the compiler, string lengths and the network.

7.3 How to calculate (show this in your journal)

```
Transaction fee (ETH) = Gas used x Gas price (gwei) / 1,000,000,000

Example: Gas used = 130,000 ; Gas price = 1.5 gwei
Fee = 130,000 x 1.5 / 1,000,000,000
    = 195,000 / 1,000,000,000
    = 0.000195 ETH
```

Under EIP-1559 the gas price is base fee + priority fee. Etherscan shows base fee (burned), max fee and priority fee separately. For the journal, effective gas price × gas used = total fee.

7.4 Observations you can write (conclusion)

- Deployment costs the most gas because the whole bytecode is stored on-chain.
- Adding new data costs more than updating existing data because new storage slots are created.
- Reading data with view functions is free.
- Gas used depends on work done; the fee also depends on network congestion (gas price).
- Failed transactions still cost the gas consumed.

Part 8: Practical 5, Survey report on blockchain types and use cases

> `05_SurveyReport.md` was not uploaded. Your README says it contains blockchain types (public, private, consortium, hybrid), a 50-respondent sample survey, and real deployments in payments, trade finance, KYC, CBDC and audit. Below is the theory so you can fill and explain it. Put your own name and roll number where the file asks.

8.1 Types of blockchain

Type	Who can join / read?	Who validates?	Speed / privacy	Examples
Public	Anyone	Anyone (miners / validators)	Slower, fully transparent	Bitcoin, Ethereum

Type	Who can join / read?	Who validates?	Speed / privacy	Examples
Private	Only invited participants of one organisation	One organisation controls it	Fast, private; more centralised	Internal company ledgers; Hyperledger Fabric used by a single firm
Consortium (federated)	Group of pre-selected organisations	A set of known member organisations	Fast, shared control among members	Hyperledger Fabric networks, R3 Corda, Quorum-based bank networks
Hybrid	Mix: some data public, some private	Rules decide what is public	Flexible; public verifiability with private data	XDC Network, Dragonchain (commonly cited examples)

Permissionless = anyone can join (public). Permissioned = need approval (private/consortium).

8.2 Real-time use cases (check latest status before quoting examples)

Area	Problem today	How blockchain helps	Examples
Payments / remittance	Cross-border transfers are slow and costly through many banks	Direct settlement on a shared ledger, near-real-time, fewer intermediaries	Ripple (XRP Ledger), stablecoins, bank blockchain payment networks
Trade finance	Paper documents (letters of credit, bills of lading) cause delays and fraud	Shared tamper-proof record of documents; smart contracts release payment when conditions are met	Marco Polo, Contour, earlier platforms such as we.trade and TradeLens (some have since closed)

Area	Problem today	How blockchain helps	Examples
KYC (Know Your Customer)	Each bank repeats identity checks	Verified identity data shared once, with customer consent	Consortium KYC utilities, self-sovereign identity pilots
CBDC (Central Bank Digital Currency)	Cash usage decline; need for digital sovereign money	Central bank issues digital currency on a controlled ledger	India's Digital Rupee (e-rupee) pilots, China's e-CNY, Bahamas Sand Dollar
Audit	Manual sampling, records can be altered	Immutable time-stamped records; auditors can verify all transactions	Blockchain audit trails in supply chain and finance

8.3 Sample survey method (50 respondents)

A survey collects opinions from people to measure awareness and acceptance.

- **Respondents:** 50 students/staff/professionals. Record age group and occupation.
- **Sample questions:** Have you heard of blockchain? (Yes/No); Which crypto wallet do you know?; Do you think blockchain improves trust in payments? (Agree/Neutral/Disagree); Which sector benefits most (Payments, Trade finance, KYC, CBDC, Audit)?; Which type of blockchain is best for banks?
- **Results:** count answers, convert to percentages (count ÷ 50 × 100), show in a table and a bar or pie chart.
- **Conclusion:** write 3 to 4 lines, e.g. most respondents are aware of blockchain but few have used it; permissioned blockchains are preferred for banking; the biggest concerns are regulation and scalability.

> **Exam point:** If the file has a ready sample survey table, use its numbers but be ready to explain how percentages were calculated. Add your name and roll number as the README says.

Part 9: Practical 6, Business network on Hyperledger Fabric

> The folder `06_HyperledgerBusinessNetwork/` was not uploaded, so I cannot check its chaincode. The theory is standard. The commands are the commonly used Fabric `test-network` commands; always prefer the exact commands in your folder's own README, because versions change.

9.1 Theory

Hyperledger Fabric is an open-source permissioned (enterprise) blockchain framework hosted by the Linux Foundation. Only known, authenticated organisations join. It has no cryptocurrency or mining; it uses an ordering service for consensus.

Term	Meaning
Organisation (Org)	A company in the network, e.g. Org1, Org2.
Peer	Node that holds a copy of the ledger and runs chaincode.
Orderer (ordering service)	Orders transactions into blocks (Raft in recent versions). Does not execute contracts.
Channel	A private sub-network; each channel has its own ledger shared only by its members.
Ledger	Two parts: the blockchain (log of transactions) + world state (current values, kept in a database).
Chaincode	Fabric's name for a smart contract (Go, JavaScript or Java). Yours is a JavaScript business registry with create, read, update, delete, list.
MSP (Membership Service Provider)	Defines identities and who belongs to which organisation (uses certificates).
CA (Certificate Authority)	Issues identity certificates.
Endorsement policy	Rule such as both Org1 and Org2 must approve a transaction.

Transaction flow: Execute, Order, Validate

1. **Propose:** the client application sends a transaction proposal to endorsing peers.
2. **Endorse:** peers run the chaincode (simulation) and sign the result.
3. **Order:** the client sends the endorsed transaction to the ordering service, which makes a block.
4. **Validate and commit:** peers check the endorsement policy and conflicts, then add the block to the ledger and update the world state.

Fabric vs Ethereum

Point	Ethereum (public)	Hyperledger Fabric
Access	Open to everyone	Permissioned, known identities
Consensus	Proof of Stake	Ordering service (Raft)
Currency	ETH needed for gas	No native coin required
Contract language	Solidity	Go / JavaScript / Java chaincode
Privacy	All data public	Channels and private data collections

9.2 What the Ubuntu lab needs

Open the Terminal with Ctrl + Alt + T and check each tool:

```
docker --version
docker compose version
node -v
npm -v
go version
git --version
curl --version
```

If a tool says `command not found`, ask the lab incharge whether it is preinstalled somewhere or whether you may install it. On a personal Ubuntu PC the usual commands are:

```
sudo apt update
sudo apt install -y git curl docker.io docker-compose-v2 golang-go nodejs npm
sudo systemctl enable --now docker
sudo usermod -aG docker $USER      # then log out and log in again
```

> **Careful:** These need sudo (administrator) rights and internet. Lab PCs are often locked, in which case Fabric is usually set up for you. Package names can differ between Ubuntu versions.

9.3 Typical run steps (follow your folder's README first)

```
# 1. Download Fabric samples and binaries (needs internet, takes several
minutes)
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-
fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

# 2. Start the test network and create a channel
cd fabric-samples/test-network
./network.sh down
./network.sh up createChannel -c mychannel -ca

# 3. Package, install and commit your chaincode
#    (replace the path with YOUR chaincode folder from
06_HyperledgerBusinessNetwork)
./network.sh deployCC -ccn basic -ccp /full/path/to/your/chaincode -ccl
javascript

# 4. After you finish, clean up
./network.sh down
```

After deployment, the folder's README shows `peer chaincode invoke` and `peer chaincode query` commands (with environment variables for Org1) to create, read, update, delete and list records. Use those exact commands.

Part 10: Exam day plan and troubleshooting

10.1 Before you enter the lab

- Wallet created, Sepolia selected, at least 0.05 test ETH available.
- Both `.sol` files ready on a pen drive or email.
- Remix tested once with both contracts.

- Revise Part 1, Part 5.7 and Part 11 (viva).

10.2 In the lab: order of work

1. Open the browser and MetaMask; check network = Sepolia and balance > 0.
2. Open Remix. Add the contract file the examiner asks for. Compile. (2 min)
3. Deploy via Injected Provider - MetaMask. Screenshot the terminal and the contract address.
4. Run the test table for that practical. Screenshot each major step.
5. For Practical 4: record gas used, gas price and fee for each operation (Part 7).
6. Show the transaction on Sepolia Etherscan if the examiner asks.
7. Lock MetaMask or clean the browser before you leave.

10.3 What to put in your journal / write-up

- Title, aim, theory (short), software used (MetaMask, Remix, Sepolia).
- Algorithm / steps (use Part 5.5 and 6.4 wording).
- Program listing (paste the code).
- Output screenshots: compile success, deployment, each function call, events.
- Gas and fee table (Practical 4) with the formula and one worked calculation.
- Conclusion: 2 lines about what you learned.

10.4 Troubleshooting table (all practicals)

Symptom	Likely cause	What to do
Remix does not show MetaMask option / nothing happens	Extension disabled or popup blocked	Click the fox icon, unlock MetaMask; allow popups; refresh Remix
Network shows Ethereum Mainnet	Wrong network	Switch MetaMask to Sepolia, refresh Remix and reconnect
Deploy fails with insufficient funds	0 test ETH	Use a faucet or ask a friend to send 0.05 Sepolia ETH

Symptom	Likely cause	What to do
Transaction stuck pending	Low gas price / congestion	Wait a few minutes; in MetaMask use Speed up if shown
Remix says Gas estimation failed	The function will revert (over-withdraw, duplicate id)	Expected for negative tests; screenshot it
Compile red error	Typo while pasting or old compiler	Re-copy the whole file; compiler 0.8.20+
Pasted code has strange quotes	Copied from a chat or document that changed quotes	Copy only from the original `.sol` file in the Text Editor
Faucet says not eligible	Faucet needs an account or balance	Try the other faucet or ask classmates
Contract vanished from Deployed Contracts	Page refreshed	Use At Address with the saved contract address

Part 11: Viva questions with answers

1. **What is a blockchain?** A distributed, tamper-evident ledger of blocks linked by hashes and maintained by many nodes through consensus.
2. **What is a hash and why is it important?** A fixed-length fingerprint of data. Any change in data changes the hash, so it links blocks and detects tampering.
3. **What is a smart contract?** A program stored on the blockchain that executes automatically and cannot be altered after deployment.
4. **What is the EVM?** Ethereum Virtual Machine, the runtime that executes contract bytecode on every node.
5. **What is gas? Why is it needed?** A unit measuring computational work. Users pay for it, which pays validators and prevents spam and infinite loops.
6. **How is the transaction fee calculated?** Fee = Gas used × Gas price. With gas price in gwei, divide by 10^9 to get ETH.
7. **Difference between gas limit and gas used?** Limit is the maximum you allow; used is what was actually consumed. Unused gas is refunded.

8. **What is Sepolia? Why use a testnet?** A test network of Ethereum with free, valueless ETH. It lets us test without risking real money.
9. **What is a faucet?** A website that gives free test ETH to developers.
10. **Public key vs private key vs seed phrase?** The public address is shareable. The private key signs transactions and must be secret. The seed phrase (12 words) regenerates private keys.
11. **What is MetaMask?** A browser-extension wallet that stores keys, signs transactions and connects websites to Ethereum.
12. **What is Remix?** A browser-based IDE to write, compile, deploy and test Solidity contracts.
13. **What does payable do?** Allows a function or address to receive ETH.
14. **What are msg.sender and msg.value?** msg.sender is the caller's address; msg.value is the amount of wei sent with the call.
15. **What is re-entrancy?** An attack where a malicious contract calls back into the victim before its state is updated, draining funds. Prevented by updating state before external calls (Checks-Effects-Interactions) or reentrancy guards.
16. **Why msg.sender and not tx.origin?** tx.origin is the original external account in a call chain; relying on it enables phishing through intermediate contracts.
17. **Why use a mapping in the bank contract?** Constant-time lookup of a balance by address, without loops.
18. **Explain struct and array in the student contract.** A struct groups id, name, course and age into one type. The dynamic array stores many such records.
19. **Why does indexOf store position + 1?** A mapping returns 0 for missing keys. Storing +1 lets 0 mean not found.
20. **How does removeStudent work in O(1)?** It copies the last element into the removed slot, updates the moved id's index, and pops the array. No shifting.
21. **Difference between receive() and fallback()?** receive: plain ETH with empty calldata. fallback: calldata that matches no function (or no receive exists).
22. **Why is a view function free?** It only reads state and is executed locally by one node; it creates no transaction.
23. **What do memory, storage and calldata mean?** storage: permanent chain data; memory: temporary working data; calldata: read-only call input.
24. **What is an event?** A log entry emitted by a contract, cheap to store and readable by apps and explorers.

25. **What happens when a transaction reverts?** All state changes are undone; gas used is still paid.
26. **Public vs permissioned blockchain?** Public: anyone can join, e.g. Ethereum. Permissioned: only approved members, e.g. Hyperledger Fabric.
27. **What replaces mining in Fabric?** An ordering service (Raft) plus endorsement by peers; there is no Proof of Work.
28. **Name use cases of blockchain.** Payments, trade finance, KYC, CBDC, audit, supply chain, healthcare records, voting.
29. **Why does storing data cost more gas than computation?** Stored data lives on every node permanently, so storage operations are priced much higher.
30. **What is the Merkle root?** A single hash summarising all transactions in a block, built by hashing pairs repeatedly into a tree.

Part 12: Glossary

Term	Meaning
ABI	Application Binary Interface: description of a contract's functions used to call it.
Block	Group of transactions linked to the previous block by hash.
Bytecode	Compiled machine-level code the EVM executes.
CBDC	Central Bank Digital Currency.
Chaincode	Smart contract in Hyperledger Fabric.
Consensus	Process by which nodes agree on the next valid block.
EOA	Externally owned account controlled by a private key.
ETH / Ether	Native currency of Ethereum.
Faucet	Site that gives free testnet coins.
Gas / Gwei / Wei	Work unit / 10^9 wei / smallest ETH unit (10^18 wei = 1 ETH).

Term	Meaning
Hash	Fixed-size fingerprint of data.
KYC	Know Your Customer identity verification.
Mainnet	The real Ethereum network with valuable ETH.
Node	A computer that stores and verifies the blockchain.
Nonce	A number used once; in PoW miners vary it to find a valid hash; in accounts it counts your transactions.
Re-entrancy	Attack where a contract is called again before it finishes updating its state.
Revert	Cancelling a transaction and undoing its changes.
Sepolia	Ethereum test network used in these practicals.
Solidity	Programming language for Ethereum smart contracts.
Testnet	Network for testing with valueless coins.
Wallet	Software that holds keys and signs transactions.

> Good luck! Do Practical 2 (wallet + test ETH) first today, because everything else depends on having Sepolia ETH.